Guía del laboratorio — OWASP Amass

Laboratorio de reconocimiento y mapeo de superficie de ataque Dominio ficticio: `lab.local` · Red: `192.168.56.0/24`

Expositor: Michael Stiven Tabares Tobón **Curso:** Seguridad de la Información — 2026-II · Ingeniería de Sistemas, UdeA

Código y descargas

https://github.com/Mista299/lab-amass-owasp-seguridad-info

Para tener el laboratorio en tu equipo:

```
git clone https://github.com/Mista299/lab-amass-owasp-seguridad-info.git
cd lab-amass-owasp-seguridad-info
```

Antes de empezar · Cómo funciona un DNS

Internet conecta **direcciones IP**, no nombres. El DNS traduce nombres a direcciones:

```
dev.lab.local  ──[pregunta]──▶  DNS  ──[respuesta]──▶  192.168.56.103
```

`dig` es el cliente de línea de comandos para preguntar. Empieza siempre con `dig +short`, que imprime solo la respuesta:

```
dig +short @192.168.56.101 dev.lab.local A
```

```
192.168.56.103
```

Un nombre que **no** existe no devuelve nada:

```
dig +short @192.168.56.101 test.lab.local A
```

Sin `+short` ves el código de estado, que es lo que importa:

Status	Significado
`NOERROR`	La consulta funcionó. Si hay respuesta, el nombre existe
`NXDOMAIN`	**No existe.** El nombre no está en la zona

Un registro no es solo nombre + IP. Los tipos que importan aquí:

Tipo	Para qué
`A`	Nombre a IPv4
`CNAME`	Alias: un nombre apunta a otro
`NS`	Qué servidores DNS son los autorizados
`MX`	Servidor de correo
`SRV`	Puerto y protocolo de un servicio

Qué le añade Amass al `dig`: hace las preguntas por ti (genera candidatos y pregunta por cada uno), las cruza (un `A` da la IP, un `MX` da el servidor de correo), y guarda el resultado como un grafo de activos con relaciones. `dig` hace una pregunta; Amass construye un inventario.

Paso 1 · Montar el laboratorio

Desde la raíz del proyecto:

```
./start.sh
```

La primera vez tarda unos 2 minutos (descarga y construye las imágenes).

Salida esperada:

```
==> Verificando requisitos
 [✓] Docker: Docker version 29.7.2
 [✓] Docker Compose: 5.5.1
 [✓] Daemon de Docker accesible

==> Construyendo las imágenes (la primera vez tarda ~2 min)
 [✓] Imágenes construidas

==> Levantando el laboratorio
 [✓] Contenedores iniciados

==> Esperando a que el motor de Amass esté listo
 [✓] Motor listo tras 1s

==> Verificando el DNS del laboratorio
 [✓] DNS del laboratorio responde (192.168.56.101:53)
 [✓] Resolución de subdominios funciona (dev.lab.local -> 192.168.56.103)
 [✓] NXDOMAIN correcto para los candidatos inexistentes
```

Para empezar de cero: `./start.sh --reset`

Paso 2 · Entrar en la máquina Amass

```
cd lab-infra/docker
docker compose -f compose.yaml exec amass bash
```

El prompt cambia. A partir de aquí estás dentro del contenedor y todos los comandos se ejecutan en la máquina Amass.

Para comprobarlo:

```
hostname        # deve mostrar algo como amass-lab-amass
which amass     # /bin/amass
```

Para salir: `exit`

Paso 3 · El escaneo

Ojo con dónde estás

Los comandos de este paso están en **dos sitios distintos**. Mira el prompt antes de copiar:

Tu prompt	Dónde estás	Qué puedes ejecutar
`usuario@equipo:~/...$`	Tu portátil	`docker compose ...`, `./start.sh`
`d18e4b7ae2c1:/data#`	Dentro de la máquina Amass	`amass ...`, `dig ...`, `sqlite3 ...`

Si intentas un comando de `docker` estando dentro del contenedor, sale `docker: command not found`. Para volver al portátil: `exit`.

Dentro de la máquina Amass

Estos comandos se ejecutan **dentro de la máquina Amass**.

Primero, comprueba que el DNS ficticio responde

```
dig +short @192.168.56.101 dev.lab.local A
```

```
192.168.56.103
```

Un nombre que no existe (el DNS responde NXDOMAIN, no hay salida):

```
dig +short @192.168.56.101 test.lab.local A
```

Versión de Amass

```
amass -version
```

```
v5.1.3
```

La enumeración

```
amass enum \
  -engine http://127.0.0.1:4000 \
  -active \
  -brute -w /wl/subdomains-lab.txt \
  -d lab.local \
  -p 80,443 \
  -timeout 3
```

Bandera	Significado
`-engine http://127.0.0.1:4000`	Habla con el motor que corre en esta máquina
`-active`	Pregunta al DNS y comprueba los puertos
`-brute -w /wl/subdomains-lab.txt`	Prueba los nombres de esa lista
`-d lab.local`	El objetivo
`-p 80,443`	Solo mira estos dos puertos
`-timeout 3`	Si deja de avanzar 3 minutos, se detiene

Tarda entre 2 y 4 minutos.

Si necesitas copiarlo en una sola línea (por ejemplo, para pegarlo en el terminal sin que se corte):

```
amass enum -engine http://127.0.0.1:4000 -active -brute -w /wl/subdomains-lab.txt -d lab.local -p 80,443 -timeout 3
```

Los resultados

```
amass subs -names -d lab.local -ip
```

```
admin.lab.local 192.168.56.105
api.lab.local 192.168.56.109
dev.lab.local 192.168.56.103
gitlab.lab.local 192.168.56.107
grafana.lab.local 192.168.56.108
jenkins.lab.local 192.168.56.107
lab.local 192.168.56.103
mail.lab.local 192.168.56.103
ns1.lab.local 192.168.56.101
old.lab.local 192.168.56.110
qa.lab.local 192.168.56.104
staging.lab.local 192.168.56.103
vpn.lab.local 192.168.56.106
www.lab.local 192.168.56.103
```

La fuerza bruta completa

Por qué hace falta un paso aparte

La enumeración de arriba descubre los subdominios **que el DNS publica** por sus registros NS, MX y SRV. En el laboratorio eso da tres nombres: `lab.local`, `mail.lab.local` y `ns1.lab.local`.

Los otros once (`dev`, `vpn`, `jenkins`…) **no aparecen** en ningún registro, y el DNS no tiene forma de anunciarlos. Solo aparecen si alguien **pregunta por ellos uno a uno**. Eso es la fuerza bruta.

El paso adicional hace exactamente eso: recorre la lista de 13 candidatos y pregunta al DNS por cada uno.

Cómo se ejecuta

Sal de la máquina Amass (`exit`) y desde la raíz del proyecto:

```
./start.sh --brute
```

Salida:

```
==> Fuerza bruta DNS · 13 candidatos · dominio lab.local
==> Paso 1/2 · Enumeración base del dominio
 [✓] Motor listo
==> Paso 2/2 · Validando cada candidato contra el DNS
    www.lab.local [✓] EXISTE
    mail.lab.local [✓] EXISTE
    dev.lab.local [✓] EXISTE
    ...
    test.lab.local [✗] descartado (NXDOMAIN)
    stage.lab.local [✗] descartado (NXDOMAIN)

==> Resultado de la fuerza bruta
    Candidatos probados : 13
    Confirmados         : 11
    Descartados         : 2
```

Esa diferencia es el punto: **la fuerza bruta no solo confirma, también descarta**. El DNS es el que decide.

> **Nota técnica:** en Amass v5.1.3 la opción `-nf` provoca un error interno cuando se le pasan varios nombres a la vez. Por eso el script los prueba de uno en uno. Es un detalle de la implementación, no de la técnica.

La lista de palabras

El archivo `lab-infra/wordlists/subdomains-lab.txt` está creado para este laboratorio. No es una lista genérica ni viene con Amass.

```
# --- Existen en la zona lab.local (11) ---
www
```

```
mail
dev
staging
qa
admin
vpn
jenkins
gitlab
grafana
api

# --- NO existen en la zona: se descartan (2) ---
test
stage
```

La zona DNS (`lab-infra/dns/zone.txt`) se diseñó primero, con activos realistas de una organización de software, y de ella se extrajeron las 11 palabras que la fuerza bruta debe encontrar.

`test` y `stage` no existen a propósito. `staging` existe pero `stage` no, y `test` es el nombre que todo el mundo probaría. Si los 13 encontraran respuesta, la demostración no probaría nada.

Comandos opcionales

Ninguno es necesario para la demostración. Sirven para entender mejor la herramienta.

Las consultas DNS que hizo Amass

Muestra cada nombre que se le preguntó al servidor DNS.

Este comando va FUERA del contenedor. Si el prompt muestra `d18e4b7ae2c1:/data#` estás dentro de la máquina Amass y ahí no existe el comando `docker` .

Primero sal:

```
exit
```

El prompt vuelve a mostrar la ruta de tu proyecto. Y desde `lab-infra/docker` :

```
docker compose -f compose.yaml logs dns | grep "query" \
  | sed 's/.*\[query\] //' | awk '{print $1}' | sort -u
```

> **Si prefieres no cambiar de sitio**, hay un atajo que funciona desde cualquier directorio y desde dentro o fuera del contenedor:
>
> `bash ./start.sh --dns`

La base de datos de activos

Amass guarda el grafo en SQLite. Dentro de la máquina Amass:

```
sqlite3 /.config/amass/asset.db "SELECT fqdn FROM fqdn ORDER BY fqdn;"
```

Los nombres guardados.

Cuántas relaciones se encontraron:

```
sqlite3 /.config/amass/asset.db "SELECT COUNT(*) FROM edge;"
```

Las tablas disponibles:

```
sqlite3 /.config/amass/asset.db ".tables"
```

La zona DNS completa

Sal de la máquina Amass (`exit`) y desde `lab-infra/docker` :

```
docker compose -f compose.yaml exec dns cat /app/zone.txt
```

Dos preguntas que suelen aparecer

"¿No es trampa buscar directo en el DNS?"

No. Es al revés: **es exactamente lo que hace Amass**.

La diferencia es quién formula la pregunta:

Tú directamente con `dig`	**Amass**
Preguntas un nombre que tú ya sabes	Genera los candidatos, pregunta y **correlaciona**
Ves un nombre o un NXDOMAIN	Ves el nombre **con su IP, su servicio y su procedencia**
No queda registro	Queda en el grafo, con TTL y confianza

Si en la demo preguntas `dig @192.168.56.101 dev.lab.local` y obtienes `192.168.56.103` , estás haciendo una consulta DNS normal. Eso es legitimo y es lo que hace la herramienta. La diferencia es que Amass hace 13 de esas, las convierte en activos con relaciones, y guarda cuáles fallaron.

De hecho, el comando `logs dns` que se ve arriba **es la bitácora de Amass**. Esa es la prueba: el servidor DNS registra cada consulta, y se ve de dónde viene cada nombre. Es la misma técnica de la vida real, pero con la posibilidad de auditar qué se preguntó.

"¿Qué es el TTL?"

TTL = Time To Live. Es cuántos segundos puede un resolver DNS guardar una respuesta antes de tener que volver a preguntarle al servidor autoritativo.

```
Servidor DNS  ──"dev.lab.local = 192.168.56.103"──►  Resolver DNS
                                                          |
                                                lo guarda 60 s (TTL)
                                                          |
                                                si alguien pregunta de nuevo,
                                                responde sin preguntar al servidor
```

En nuestro laboratorio el TTL de la zona es **300 segundos** (5 minutos) y el de la base de datos de Amass es **1440 minutos** (24 horas).

Dónde aparece en Amass:

Dato	Significado
TTL de un registro DNS	Cuánto tiempo un resolver puede cachearlo
TTL en la base de datos de Amass	Cuánto tiempo Amass da por válido un hallazgo antes de volver a verificarlo

Por qué importa en seguridad:

- Si el TTL es alto (24 h), un subdominio puede seguir resolviendo aunque lo hayas eliminado. Los activos "muertos" tardan en desaparecer.
- Si el TTL es bajo (5 min), los cambios se propagan rápido, pero Amass tiene que re-preguntar más a menudo, y eso genera más tráfico (y mas facilmente detectable).
- Un activo con TTL alto **y** ninguna fuente que lo confirme es probablemente un residuo.

La zona del laboratorio

`lab.local` tiene 32 nombres en `192.168.56.0/24`. Los que importan:

Nombres	IP	Severidad
`vpn`	192.168.56.106	Crítico
`admin`, `portal`	192.168.56.105	Alto
`jenkins`, `gitlab`	192.168.56.107	Alto
`dev`, `staging`	192.168.56.103	Alto
`qa`, `uat`, `preprod`	192.168.56.104	Alto
`grafana`, `kibana`	192.168.56.108	Medio
`api`, `beta`, `sandbox`	192.168.56.109	Medio
`old`, `backup`	192.168.56.110	Medio
`www`, `mail`	192.168.56.103	Bajo
`ns1`	192.168.56.101	Infraestructura

Además hay 8 alias CNAME (`intranet` , `secure` , `ci` , `scm` , `metrics` , `logs` , `preview` , `legacy`) y un registro SRV (`_sip._tcp`).

Amass no prioriza: entrega el inventario con su procedencia. La severidad depende del contexto, y eso lo decide el equipo de seguridad, no el script.

Atajos del script

Comando	Qué hace
`./start.sh`	Monta el laboratorio y verifica el DNS
`./start.sh --reset`	Empieza de cero
`./start.sh --enum`	Ejecuta la enumeración activa
`./start.sh --brute`	Fuerza bruta completa (confirma y descarta)
`./start.sh --results`	Muestra los resultados guardados
`./start.sh --dns`	Muestra las consultas DNS registradas
`./start.sh --check`	Verifica el estado
`./start.sh --logs`	Logs en vivo
`./start.sh --stop`	Detiene el laboratorio
`./start.sh amass <args>`	Ejecuta un comando de Amass a mano

```
./start.sh amass -version
./start.sh amass subs -names -d lab.local -ip
```

Problemas frecuentes

Síntoma	Causa	Solución
`Failed to perform a bulk transfer of assets: EOF`	Error interno de Amass v5.1.3 al pasar varios nombres con `-nf` . Mata el motor.	Usa `./start.sh --brute` , que los pasa de a uno.
`Failed to create a session with the Amass engine`	El motor no está listo	`./start.sh` espera al motor. Si persiste, `--reset` .
`Failed to obtain the BGPTools IP address`	`bgp.tools` no resuelve en el DNS del laboratorio	Verifica que está en `lab-infra/docker/zone.txt` y haz `--reset`
`flag provided but not defined: -src` / `-ip` / `-o`	Banderas de v3/v4 que ya no existen en v5	Usa `amass subs` para leer resultados
		Usa `amass subs` o `sqlite3`

Síntoma	Causa	Solución
`subcommand provided but not defined: db`	El subcomando `db` fue eliminado en v5	
La enumeración no descubre nada	Faltan las `transformations` en el config	Restaura el bloque `transformations:` de `lab-infra/configs/config.yaml`
`service "amass" is not running`	Los contenedores están detenidos	`./start.sh` los levanta
`Address already in use`	Ya hay contenedores con esos nombres	`./start.sh --stop` y vuelve a levantar
`Failed to populate the ASN cache: zero entities found`	Normal: requiere WHOIS/RDAP	Ignorar
`docker: permission denied`	Tu usuario no está en el grupo `docker`	`sudo usermod -aG docker $USER` y reinicia sesión

---

Instalar Amass en tu equipo (opcional)

No hace falta para el laboratorio. Si quieres practicar la herramienta fuera del laboratorio:

Arch Linux, con el binario oficial:

```
cd /tmp
curl -LO https://github.com/owasp-amass/amass/releases/download/v5.1.3/amass_linux_amd64.tar.gz
curl -LO https://github.com/owasp-amass/amass/releases/download/v5.1.3/amass_checksums.txt
sha256sum -c amass_checksums.txt --ignore-missing
tar xzf amass_linux_amd64.tar.gz
sudo install -m 755 amass_linux_amd64/amass /usr/local/bin/amass
```

Kali Linux, desde los repositorios:

```
sudo apt update
sudo apt install -y amass
```

Comprobar:

```
amass -version
```

> Con Amass instalado de forma nativa sí se consulta Internet, porque las fuentes OSINT son externas. En el laboratorio no.

Dos comandos pasivos contra un dominio real

Esta es la parte que sí se puede hacer en Internet sin autorización, porque no se toca el objetivo: solo se consultan fuentes de inteligencia abierta.

Ejecuta estos en tu máquina, fuera del laboratorio.

1. Reconocimiento pasivo con Amass

```
amass enum -d example.com -timeout 2
```

Consulta Certificate Transparency, Wayback, RapidDNS y otras fuentes públicas. No genera tráfico hacia el objetivo. Es la técnica de la fase de reconocimiento inicial de una auditoría, y la que la mayoría de programas de bug bounty permiten.

Tarda varios minutos: cada plugin externo espera su propio timeout.

2. Certificate Transparency sin Amass

```
curl -s "https://crt.sh/?q=%25.example.com&output=json" | jq -r '.[].name_value' | sort -u
```

Certificate Transparency es un registro público en el que las autoridades de certificado publican cada certificado que emiten. Por eso es la mejor fuente pasiva de subdominios: revela nombres que la organización no publica en ningún sitio.

Qué no hacer sin autorización

Técnica	Por qué
Fuerza bruta DNS contra un dominio ajeno	Miles de consultas a servidores que no son tuyos
Escaneo de puertos	Sondea infraestructura de terceros
Transferencia de zona (AXFR)	Aunque falle, es tráfico no autorizado

Que un dominio "no tenga nada interesante" no lo convierte en un objetivo permitido. En Colombia, escanear sin autorización es un delito informático (Ley 1273 de 2009 y Decreto 175 de 2015).

Referencias

- Sitio del proyecto: https://owasp.org/projects/amass
- Documentación oficial v5: https://owasp-amass.github.io/docs/
- Código fuente: https://github.com/owasp-amass/amass

Guía validada con OWASP Amass v5.1.3 sobre Docker.